Universidad del Cauca
Facultad de Ingeniería Electrónica y Telecomunicaciones
Programa de ingeniería de sistemas
Ingeniería de Software II Grupos A y B
Laboratorio de Ingeniería de Software II Grupos A, B y C
Francisco Obando, Daniel Paz, Juan Carlos Narvaez,. W. Libardo Pantoja Y
2026.2

Ingeniería de Software III

Teoría y Laboratorio

Entregables Proyecto de Curso - Segundo Corte 2026.1

Fecha creación: 6 Abr 2026
Última actualización: 8 Ago 2026 4:22 PM

Entregables

Para el segundo corte, se trabajarán requisitos funcionales y no funcionales del Sistema de Reserva de Citas de Piedrazul. Los requisitos 1 y 2, son los del primer corte, se pueden mejorar para la segunda iteración, por ejemplo, implementar el **autocompletado** cuando se busque un paciente por cédula.

Requisitos funcionales

Se deben implementar los siguientes requisitos funcionales de alto valor para el cliente:

1. **Yo como** agendador de citas **necesito** listar las citas médicas de un determinado médico/terapista en una fecha determinada **para** visualizar el listado y la cantidad de citas. *Contexto*: Se sugiere diseñar un sistema de búsqueda con filtros y resultados en una tabla.
2. **Yo como** paciente **necesito** agendar una cita mediante la web **para** tener una cita de manera sencilla y rápida sin tener que usar WhatsApp. *Contexto*: El paciente debe tener un registro de usuario para poder agendar una cita. El sistema debe brindar un mecanismo para hacer la cita de manera segura, usable y eficiente, mostrando las franjas disponibles para cada médico.
3. **Yo como** administrador **necesito** configurar los parámetros del sistema **para** que el agendamiento de citas autónomo funcione acorde a la disponibilidad de los médicos y terapistas de Piedrazul. *Contexto*: Se debe configurar la ventana de tiempo que se habilitarán las citas (en semanas), los días de la semana que cada médico/terapista atiende, la franja horaria de cada médico/terapista, el intervalo de tiempo (minutos) que cada médico/terapista tiene entre cita y cita.
4. **Yo como** médico/terapista/agendador **necesito** exportar las citas correspondientes a una fecha específica y a un médico o terapista determinado, en un formato de texto compatible con hojas de cálculo (por ejemplo, CSV) **para** que durante la jornada de atención haya un listado que

organizar los pacientes y su ingreso a la consulta médica.

5. **Yo como** médico/terapista/agendador **necesito** consultar las citas que de un paciente a partir de su cédula **para** conocer el historial de citas que ha tenido.

Requisitos no funcionales

1. Implementar un sistema de autenticación y autorización basado en tokens JWT. Usar un sistema externo como **Keycloak**. La gestión de usuarios y roles se puede hacer por Postman, no se requiere aún una interfaz gráfica en el frontend.
2. Testear con **OWASP ZAP** al menos dos vulnerabilidades de la aplicación web acorde a la especificación OWASP top 10. Generar un informe breve de las vulnerabilidades reportadas.

Rúbrica de evaluación

Criterio	**Excelente (5 pts)**	**Bueno (4 pts)**	**Aceptable (3 pts)**	**Insuficiente (0, 1 pts)**	**Peso (%)**
Cumplimiento de los requisitos funcionales	Implementa completamente los requisitos funcionales del primer corte	Los requisitos se implementan con pequeñas limitaciones o errores.	Los requisitos se implementan parcialmente o con errores funcionales importantes.	Los requisitos no se implementan o no cumplen las necesidades del cliente.	30%
Cumplimiento de los requisitos NO funcionales de modificabilidad, seguridad	Se aplican correctamente los principios de diseño SOLID, patrones de diseño y de arquitectura de software. Se aplica hacking ético de forma adecuada	Se aplican los principios SOLID y patrones de diseño con pequeñas limitaciones o errores	Se aplican los principios SOLID y patrones de diseño con limitaciones o errores importantes	No se aplican los principios SOLID y patrones de diseño.	30%

Documentación de arquitectura	Documento claro, estructurado, con diagramas y detalles técnicos relevantes. Repositorio Git bien organizado y documentado	Documento y repositorio adecuados, pero con pequeños aspectos mejorables.	Documento y repositorio, con aspectos importantes a mejorar.	No hay documento de la arquitectura y repositorio Git.	20%
Pruebas unitarias automatizadas	Hay pruebas unitarias a todas las clases del dominio (entidades y servicios) correctamente implementadas	Hay pruebas unitarias con pequeñas limitaciones o errores	Hay pruebas unitarias con limitaciones o errores importantes	No hay pruebas unitarias	20%

PROTOCOLO DE SUSTENTACIÓN

Para la sustentación del segundo entregable del proyecto de curso, cada equipo debe elaborar un video en **Youtube**. Importante que **todos los integrantes** del equipo participen en la sustentación. A continuación, el protocolo de sustentación:

1. Mostrar las historias de usuario con sus criterios de aceptación, prototipos de la interfaz gráfica de usuario y test de usabilidad: **2 minutos**
2. Mostrar al menos dos escenarios de calidad del atributo de seguridad (relacionados con autenticación y autorización): **1 minuto**
3. Mostrar la arquitectura y diseño de software (usar el modelo C4 y 4+1 vistas - UML): **2 minutos**
4. Mostrar las pruebas unitarias automatizadas: **1 minuto**
5. Mostrar el repositorio git con los commits de todos los integrantes del equipo y el tablero de tareas del primer sprint Scrum: **1 minuto**
6. Mostrar los patrones de diseño GoF implementados ya sea en el frontend o backend y una breve descripción de cómo y en qué contexto del problema se aplicaron: 2 **minutos**
7. Mostrar el software funcional y algunos aspectos claves de la codificación que evidencien la implementación de la arquitectura: **5 minutos**

NOTA: Los grupos que se pasen de estos tiempos (o aceleren el video), serán penalizados con -1 punto.

DOCUMENTO DE ARQUITECTURA

El documento de arquitectura debe tener estas partes:

- Portada
- Introducción breve al documento
- Historias de usuario (con criterios de aceptación) implementadas en la segunda iteración
- Prototipos de la interfaz de usuario y un test de evaluación de usabilidad
- Pantallazo de planificación de tareas del Sprint 2: Jira, Trello.
- Escenarios de calidad del atributo de seguridad: contexto, estímulo, respuesta, medición de calidad, resultado esperado.
- Arquitectura y diseño de software usando el modelo C4 y 4+1 vistas.
- Listado de patrones de diseño GoF implementados ya sea en el frontend o backend y una breve descripción de cómo y en qué contexto del problema se aplicaron.
- URL del video de YouTube
- URL del repositorio GIT